SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:100**

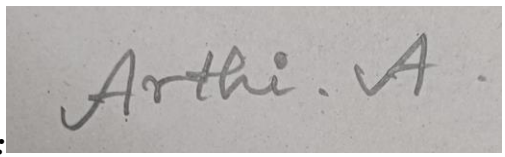
Annexure A

Debugging or Optimization task

Code of Conduct:

*I **ARTHI A – 192525152** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

lock(R1)

wait(100 ms)

lock(R2)

critical section

unlock(R2)

unlock(R1)

Process P2:

lock(R2)

wait(100 ms)

lock(R1)

critical section

unlock(R1)

unlock(R2)

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Q6. Producer–Consumer Debugging (Semaphore Error)

A producer-consumer application occasionally hangs after running for several minutes.

Semaphore empty = 5

Semaphore full = 0

Semaphore mutex = 1

Producer:

wait(mutex)

wait(empty)

insert(item)

signal(full)

signal(mutex)

Consumer:

wait(mutex)

wait(full)

remove(item)

signal(empty)

signal(mutex)

The program freezes even though both producer and consumer threads are alive.

1. Identify the synchronization error causing the hang.
2. Explain why the semaphore ordering is incorrect.
3. Suggest the correct semaphore sequence to eliminate the problem.

(10 Marks)

Q7. Thread Performance Debugging

A web server creates one new thread for every incoming client request.

Performance statistics:

- Average requests/sec = 15,000
- Average thread creation time = 0.4 ms
- CPU utilisation = 92%
- Memory usage continuously increases

Developers notice that the server becomes slow after several hours.

1. Identify the thread-management problem.

2. Explain why the current multithreading model causes performance degradation.

3. Propose two optimisations to improve scalability.

(10 Marks)

Q8. Banker's Algorithm Debugging

Available Resources:

A B C = 2 1 1

Allocation:

P1: 1 0 1

P2: 0 1 0

P3: 1 1 1

Maximum:

P1: 2 1 2

P2: 1 2 1

P3: 3 2 2

Process P2 requests (1,1,1).

1. Calculate the Need matrix.

2. Determine whether the request can be granted immediately using Banker's Algorithm.

3. If not, identify the unsafe condition and explain why granting the request could lead to deadlock.

(10 Marks)

Q9. Deadlock Detection and Recovery Debugging

A database server reports that four transactions are waiting for resources.

Resource Allocation Graph:

• T1 waits for R2

• T2 waits for R3
• T3 waits for R4
• T4 waits for R1
• R1 allocated to T1
• R2 allocated to T2
• R3 allocated to T3
• R4 allocated to T4

The database becomes completely unresponsive.

1. Identify whether a deadlock exists.
2. Explain how deadlock detection identifies this situation.
3. Suggest two recovery techniques and justify which one minimises system disruption.

(10 Marks)

Q10. Monitor-Based Synchronization Debugging

A monitor is used to synchronize access to a shared printer.

```
monitor Printer
{
 boolean busy = false;

 procedure Print()
 {
   if(busy)
      wait();

   busy = true;

   // Printing

   busy = false;
```

```
 }
}
```

Users report that some print jobs never execute even though the printer becomes free.

1. Identify the logical error in the monitor implementation.
2. Explain why waiting processes are never awakened.
3. Modify the monitor logic to ensure correct synchronization and fairness.

(10 Marks)

DEBUGGING AND OPTIMIZATION

Introduction

This report analyses ten debugging and optimization scenarios related to operating-system process management, CPU scheduling, memory management, synchronization, thread management, and deadlock handling. Each question identifies the root cause, explains the relevant operating-system concept, and recommends a correction or optimization. The aim is to improve CPU utilization, fairness, responsiveness, and overall system performance while preserving correctness.

Assumptions: The pseudo-code is treated as illustrative operating-system or concurrent-programming code. Exact behavior can depend on the scheduler, lock and condition-variable APIs, workload, and resource model. Where a question's data is insufficient to prove a conclusion, that limitation is stated explicitly.

Q1. Deadlock Debugging (Code Trace)

1.1 Root cause

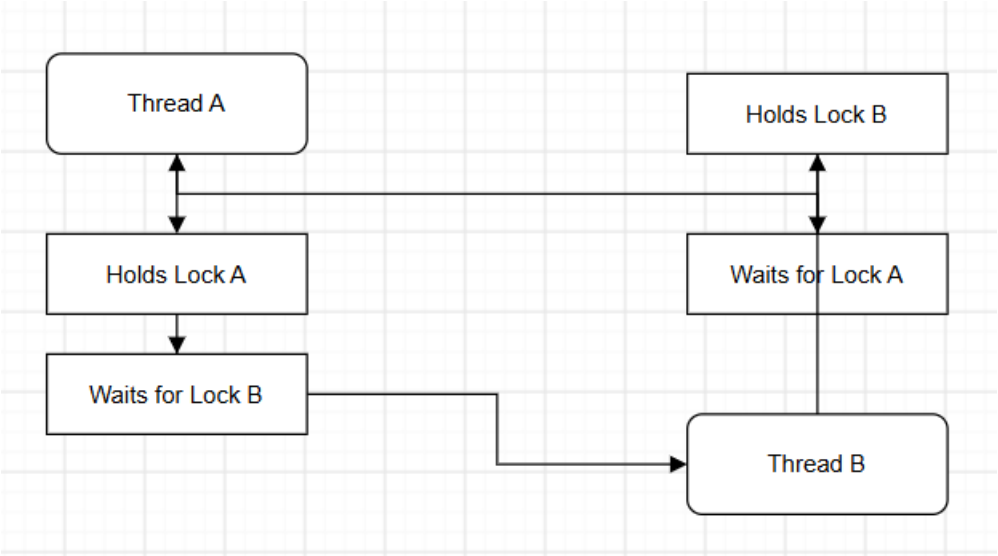
The processes acquire the same two locks in opposite orders. P1 locks R1 and then requests R2, while P2 locks R2 and then requests R1. If P1 holds R1 and P2 holds R2 at the same time, P1 waits for R2 and P2 waits for R1. Each process is waiting for a resource held by the other, so neither can proceed.

This situation satisfies the four Coffman conditions: mutual exclusion (the locks are exclusive), hold and wait (each process holds one lock while requesting another), no pre-emption (a lock is not forcibly taken away), and circular wait (P1 waits for P2 while P2 waits for P1). The conflicting lock order creates the circular-wait condition.

1.2 Optimization

Use a single, consistent global lock order in every process. For example, always acquire R1 before R2 and release them in reverse order. P2 should therefore acquire R1 first and then R2, matching P1. This removes circular wait without changing the work performed in either critical section. In production code, use the language's supported lock APIs and ensure locks are released on all exit paths, for example with scoped-lock or try/finally mechanisms.

```
Process P1: lock(R1); lock(R2); critical section; unlock(R2);
unlock(R1)\nProcess P2: lock(R1); lock(R2); critical section; unlock(R2);
unlock(R1)
```



A timed lock attempt with rollback can be an alternative, but a consistent lock hierarchy is usually simpler and avoids repeated retries. Merely adding or changing the 100 ms delays does not solve the underlying deadlock risk.

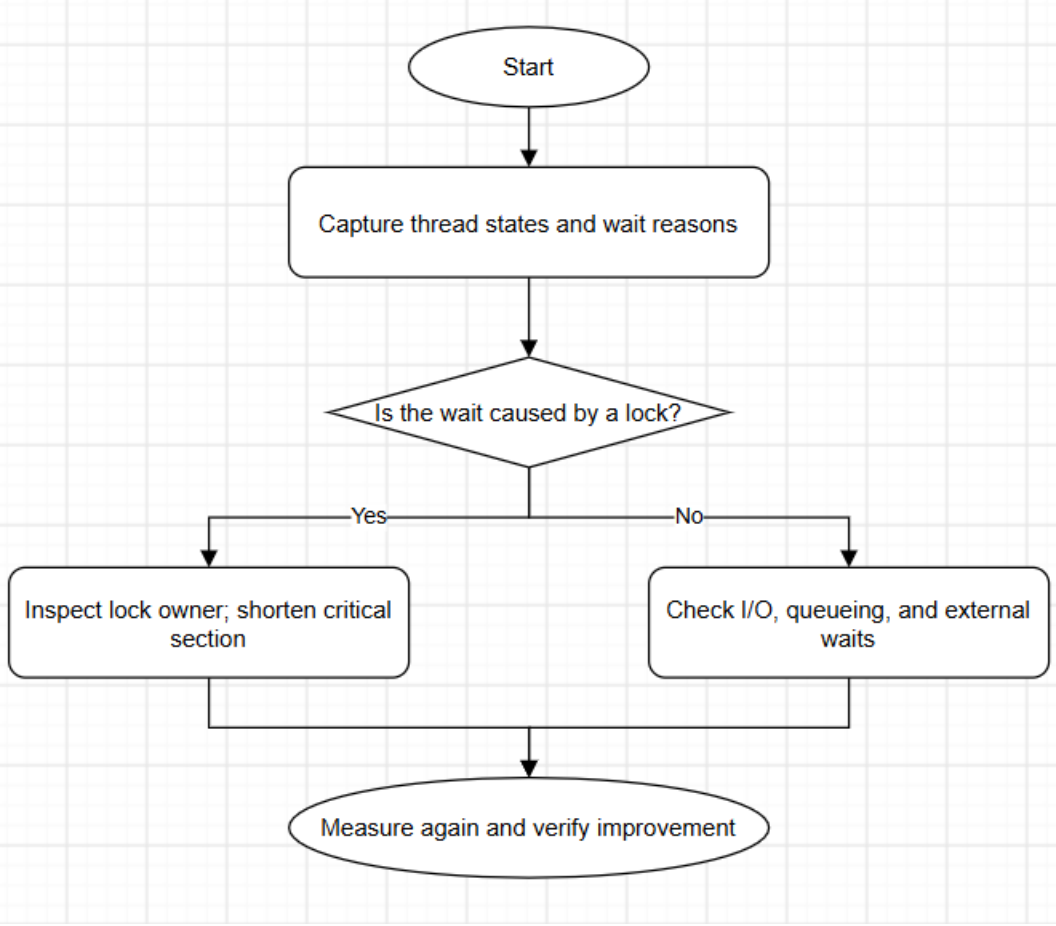
Q2. CPU Utilisation Problem

2.1 Diagnosis

A process repeatedly moving from ready to waiting and back may be blocking frequently, possibly because it is waiting for I/O, a lock, a semaphore, a condition variable, or another event. A ready process is eligible to run; a waiting process is blocked and cannot use the CPU until its event completes. If the CPU is only 30% utilized, the scheduler may have too few runnable tasks at many moments, or tasks may be blocking unnecessarily. The transition sequence alone does not prove which event is responsible.

2.2 Debugging and optimization

Inspect scheduler traces and blocking-call profiles to identify the event that causes each wait. Separate legitimate I/O waits from avoidable waits such as lock contention, overly frequent polling, serialized access to shared data, or synchronous operations on a worker thread. Measure ready-queue length, CPU run-queue occupancy, I/O latency, lock wait time, and time spent blocked.



System-level improvements include using asynchronous or non-blocking I/O for high-concurrency services, reducing lock contention, batching small I/O operations, and tuning the thread or process pool to the workload. If runnable work exists but is not scheduled promptly, inspect scheduler priorities and CPU affinity. Do not simply increase the number of threads without measurement, because excess threads can increase contention and context-switch overhead.

Q3. Memory Bottleneck Debugging

3.1 Diagnosis

The listed nominal memory footprint is approximately 5 MB of code + 150 MB of heap + 25 MB of thread stacks = 180 MB, excluding libraries, allocator overhead, mapped files, thread-local data, and other process memory. This is below 2 GB of physical RAM if that RAM is available to this process and the working set is resident. Therefore, the figures alone do not prove that total memory capacity is exhausted.

Frequent page faults may instead indicate a working set larger than the memory effectively available to the process, poor locality, random access to a large heap, memory-mapped-file activity, allocation churn, or stack/heap access patterns that continually touch new pages. Major page faults that require storage I/O are usually more expensive than minor faults that can be handled without disk reads.

3.2 Two optimizations

Optimization 1 — improve locality and reduce allocation churn. Profile page-fault counts and memory-access patterns; reuse buffers or objects where appropriate, use compact data structures, and process large datasets in cache-friendly blocks. Avoid repeatedly allocating and discarding large objects.

Optimization 2 — right-size thread and memory usage. Create threads only when useful, use a bounded worker pool, and set stack sizes to appropriate values supported by the runtime and workload. A 1 MB stack per thread is not automatically excessive, but 25 stacks reserve about 25 MB before other overhead. Also check for leaks, excessive resident-set growth, and the process's actual memory limit.

Validate changes using major/minor page-fault counters, resident-set size, allocation profiles, and throughput/latency measurements. Increasing RAM or changing paging settings should be considered only after identifying the actual cause.

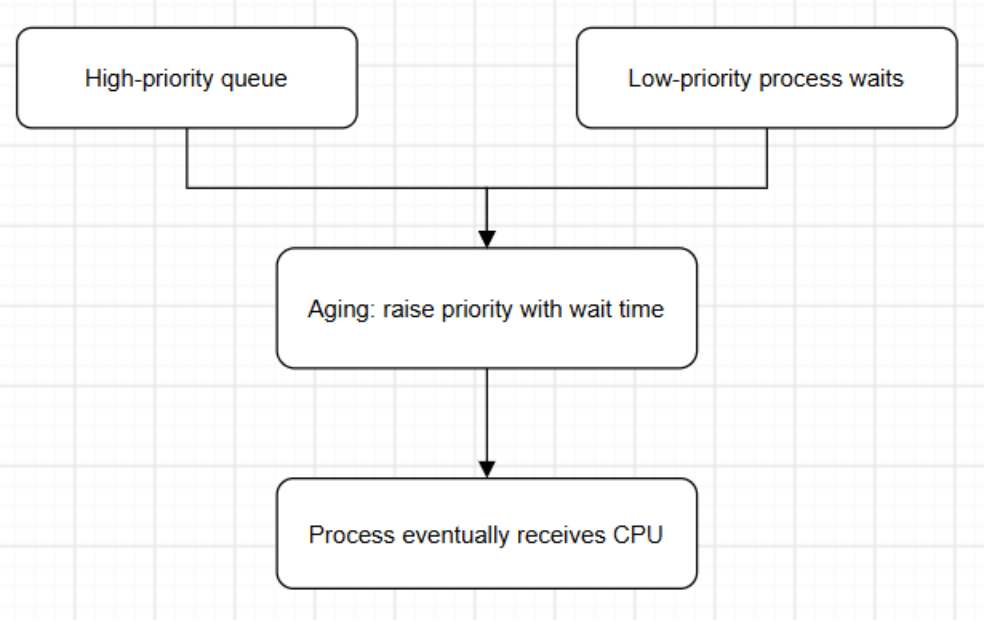
Q4. Starvation Debugging

4.1 Root cause

P5 is in Q3, the lowest-priority queue. If the multilevel queue scheduler always selects work from Q1 whenever Q1 has runnable processes, and then Q2 whenever Q2 has runnable processes, a sustained stream of higher-priority work can prevent Q3 from running. This is starvation caused by strict priority between queues, not necessarily by Q3's FCFS policy itself.

4.2 Fairness optimization

Introduce aging or a fair CPU-share policy. Aging gradually raises the effective priority of a process that has waited for a long time. Alternatively, assign each queue a guaranteed share of CPU time or use periodic priority boosts so that Q3 receives service even during heavy interactive traffic. A multilevel feedback queue can also move tasks between queues based on observed behavior, provided its rules prevent indefinite postponement.



Keep Q1 and Q2 responsive while guaranteeing a minimum service share to Q3. Monitor per-queue CPU time and maximum waiting time to confirm that the fix improves fairness without causing unacceptable interactive latency.

Q5. Context-Switch Overhead Debugging

5.1 Diagnosis

A 1 ms time quantum is short relative to the stated 2 ms average CPU burst. A CPU-bound thread may therefore be pre-empted frequently, and a high rate of I/O blocking and wake-ups can add further switches. Context switches save and restore execution state and can disturb CPU caches and translation lookaside buffers, so excessive switching consumes time that could otherwise execute application work.

However, the stated figures do not by themselves explain exactly 20,000 context switches per second. The observed rate may include many threads, I/O wake-ups, synchronization events, and scheduler activity. Profiling is needed to identify the main source.

5.2 Two optimizations

Optimization 1 — tune scheduling quantum and policy. Test a longer quantum where latency requirements permit, or use an appropriate scheduler policy that balances responsiveness with throughput. Benchmark tail latency as well as throughput; increasing the quantum indiscriminately can harm interactive response.

Optimization 2 — reduce unnecessary blocking and wake-ups. Batch small I/O requests, use asynchronous I/O where appropriate, avoid busy polling and excessive fine-grained synchronization, and use event-driven handling or bounded worker pools for high-concurrency I/O workloads.

Track context switches per request, CPU time in kernel mode, run-queue length, voluntary versus involuntary switches, throughput, and response-time percentiles before and after changes.

Q6. Producer–Consumer Debugging (Semaphore Error)

6.1 Synchronization error

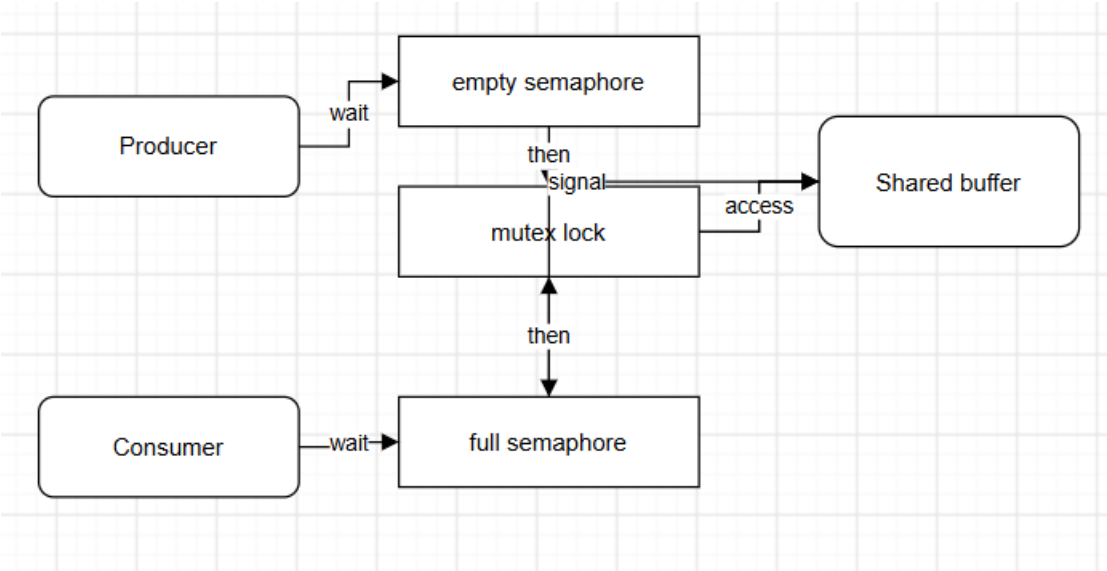
The producer and consumer acquire mutex before waiting on empty or full. This can deadlock when the producer holds mutex while waiting for an empty slot, or the consumer holds mutex while waiting for a full slot. The thread that could change the relevant count cannot enter the critical section because mutex is already held by the waiting thread.

For example, if the buffer is empty, the consumer can acquire mutex and then block on wait(full), retaining mutex. The producer cannot acquire mutex to insert an item and signal full. Both threads remain alive but neither can make progress.

6.2 Correct semaphore ordering

Wait on the resource-count semaphore before acquiring the mutex. After reserving a slot or item, acquire mutex to modify the buffer, then release mutex and signal the opposite count. This ensures a thread waiting for an item or free slot does not hold the buffer mutex.

```
Producer:\n    wait(empty)\n    wait(mutex)\n    insert(item)\n
signal(mutex)\n    signal(full)\n\nConsumer:\n    wait(full)\n
wait(mutex)\n    remove(item)\n    signal(mutex)\n    signal(empty)
```



The buffer operations must occur only while mutex is held. The count semaphores must correctly represent available slots and items, and error paths must not leave a semaphore permanently acquired. This ordering eliminates the described deadlock under the standard bounded-buffer semaphore model.

Q7. Thread Performance Debugging

7.1 Root cause

Creating one operating-system thread per request can cause excessive thread creation/destruction, stack-memory reservation, scheduler overhead, and resource pressure. At 15,000 requests per second and 0.4 ms of thread-creation time per request, thread creation alone

represents about 6 CPU-seconds of creation work per second if that time is CPU time and is additive across requests. In practice, the measured 0.4 ms may include wall-clock delay rather than pure CPU time, so profiling is required.

A 92% CPU utilization rate leaves limited headroom. As thread counts rise, scheduling, memory allocation, synchronization, and cache effects can worsen. Continuously increasing memory usage may also indicate a leak, unbounded queue, retained request state, or threads that are not exiting.

7.2 Two optimizations

Optimization 1 — use a bounded thread pool. Reuse a controlled number of worker threads and place incoming work in a bounded queue with backpressure. Select pool size based on CPU count, blocking behavior, and latency measurements rather than creating a thread for every request.

Optimization 2 — use asynchronous/event-driven I/O for large numbers of mostly waiting connections. Keep connection state lightweight and dispatch CPU-intensive work to a bounded executor. Add request timeouts, cancellation, and queue limits so overload does not turn into unbounded memory growth.

Also profile heap retention and thread counts over time to identify leaks. Evaluate throughput, p95/p99 latency, memory footprint, queue depth, and rejected or timed-out requests under load.

Q8. Banker's Algorithm Debugging

8.1 Calculate the Need matrix

Need = Maximum − Allocation, calculated element by element.

```
              Allocation      Maximum        Need\n             A  B  C
A  B  C        A  B  C\nP1           1  0  1         2  1  2        1  1
1\nP2           0  1  0         1  2  1        1  1  1\nP3           1  1
1         3  2  2        2  1  1
```

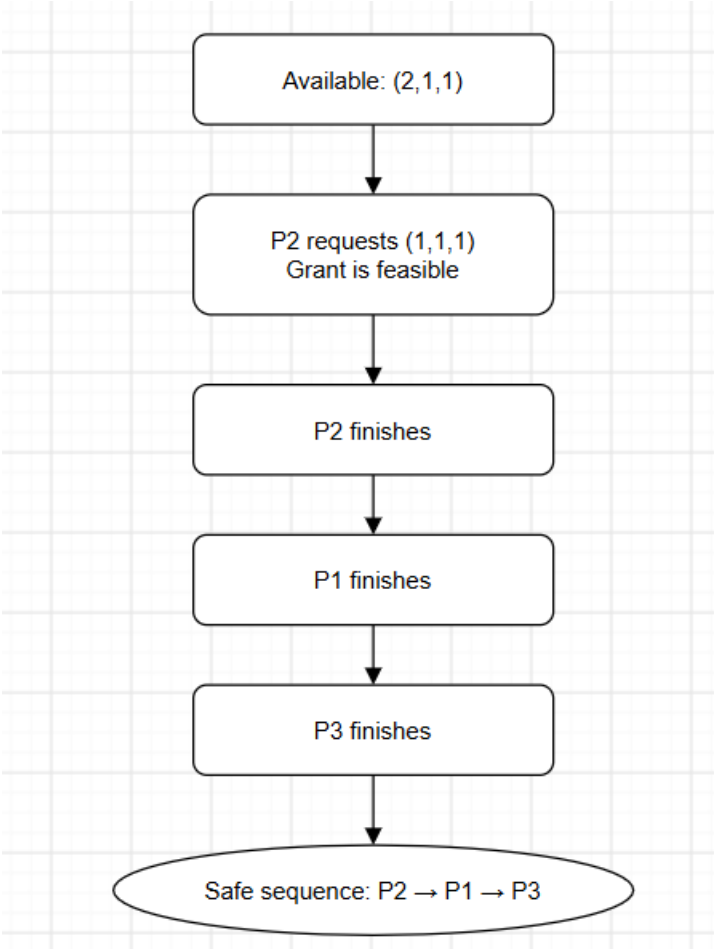
8.2 Check P2 request (1,1,1)

The request must first satisfy Request ≤ Need and Request ≤ Available. For P2, Need is (1,1,1), so the request is within its remaining declared need. However, Available is (2,1,1), and (1,1,1) is component-wise no greater than (2,1,1). The request passes both preliminary checks.

Tentatively grant the request. New Available = (2,1,1) − (1,1,1) = (1,0,0). P2's new Allocation becomes (1,2,1), and its new Need becomes (0,0,0). P2 can finish immediately and release its allocation, returning (1,2,1) to the available pool. Work then becomes (2,2,1). P1 has Need (1,1,1), so P1 can finish and release (1,0,1), giving Work = (3,2,2). P3 has Need (2,1,1), so it can also finish.

8.3 Decision and explanation

The request can be granted immediately under Banker's Algorithm because the tentative state is safe: a safe sequence is P2 → P1 → P3. No unsafe condition is produced by this request. The question's "if not" clause does not apply to the supplied values. This conclusion assumes the stated Available vector is the actual current availability before the request and that the maximum claims are valid.



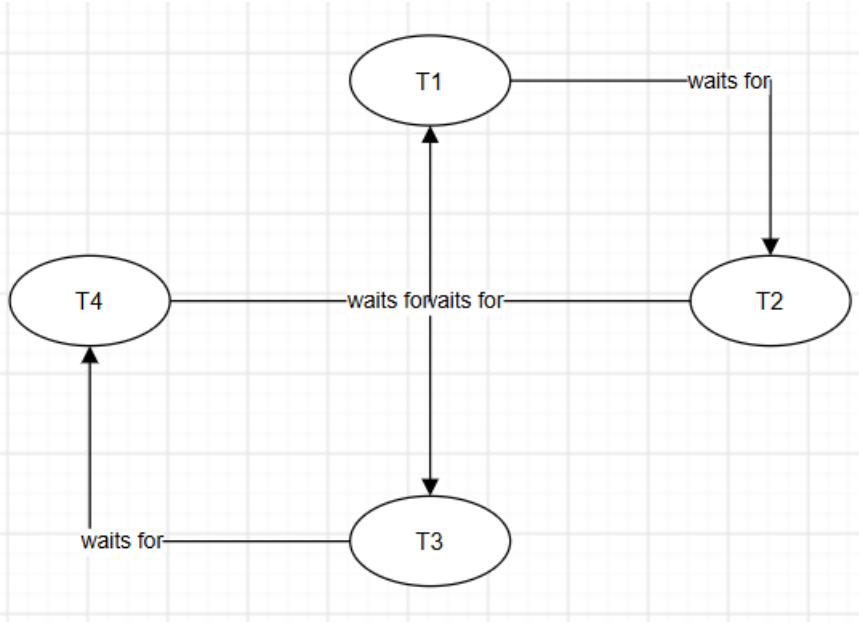
Q9. Deadlock Detection and Recovery Debugging

9.1 Does a deadlock exist?

Yes. The allocation and wait relationships form a cycle: T1 waits for R2, which is allocated to T2; T2 waits for R3, allocated to T3; T3 waits for R4, allocated to T4; and T4 waits for R1, allocated to T1. Thus T1 → R2 → T2 → R3 → T3 → R4 → T4 → R1 → T1 is a circular wait. Assuming each resource is a single-instance, non-pre-emptible resource and the transactions cannot release their held resources until they proceed, the cycle confirms deadlock.

9.2 Detection

A deadlock detector examines the resource-allocation graph or a wait-for graph. In the wait-for graph, each transaction is a node and an edge Ti → Tj means Ti is waiting for a resource held by Tj. The edges are T1 → T2, T2 → T3, T3 → T4, and T4 → T1. A directed cycle is found, so the detector reports a deadlock under the single-instance assumption. For multiple instances of a resource type, a cycle alone is not always sufficient; a suitable detection algorithm must account for instance counts and outstanding requests.



9.3 Recovery options

Technique 1 — select a victim transaction and roll it back. Choose a transaction based on rollback cost, work already completed, resources held, priority, and whether it has been rolled back before. Rolling back one carefully selected victim can release its resources and allow the remaining transactions to proceed. Use checkpoints or transaction logs to restore consistency.

Technique 2 — abort all transactions involved in the cycle, or abort them one at a time until the cycle is broken. Aborting all four is straightforward but causes more lost work and greater disruption. Incremental victim selection generally minimizes disruption because it may require rolling back only one transaction, followed by re-running deadlock detection. The database must ensure rollback is atomic and that retries do not cause starvation.

Q10. Monitor-Based Synchronization Debugging

10.1 Logical error

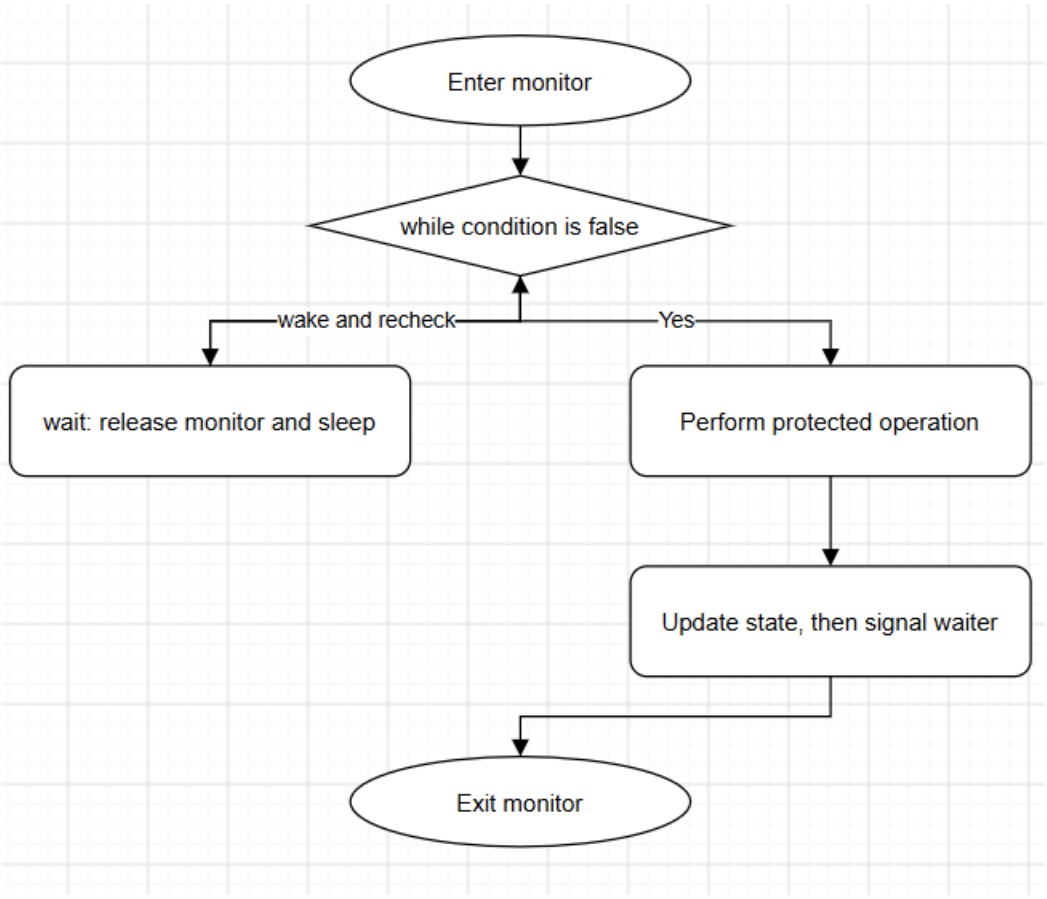
The monitor uses if (busy) wait(), but it never signals waiting processes when the printer becomes free. Also, condition-variable wait operations normally release the monitor lock while waiting and reacquire it before returning; the code must follow the semantics of the specific monitor or condition-variable API. An if check is insufficient because a process may wake after another process has already taken the printer, so the condition must be checked again in a loop.

10.2 Why jobs remain blocked

When busy is true, a caller waits. After printing, the current caller sets busy to false but does not notify a waiting caller. Therefore, a waiting thread may remain asleep even though the printer is available. If the API uses condition variables, notification is required; if it uses Hoare-style monitors or another signaling model, the exact operation differs but the missing wake-up remains the central defect.

10.3 Corrected monitor logic

```
monitor Printer {\n    boolean busy = false;\n    condition available;\n\n
procedure Print(job) {\n        while (busy) {\n
wait(available);\n        }\n        busy = true;\n        // Perform
printing while ownership is held.\n        busy = false;\n
signal(available);\n    }\n}
```



The loop rechecks the condition after waking, and signal(available) notifies a waiting caller when the printer becomes free. In a real implementation, ensure the monitor lock is released and reacquired according to the condition-variable API, and use try/finally or equivalent cleanup so busy is reset and waiters are notified even if printing fails.

To guarantee fairness, a condition variable alone may not guarantee FIFO ordering. Maintain an explicit FIFO queue of waiting jobs or use a fair semaphore/lock supported by the runtime. The next job should be selected from the queue, and cancellation or failure must remove or advance the queued job safely.

Conclusion

The scenarios demonstrate that performance problems often arise from avoidable blocking, inconsistent resource ordering, unfair scheduling, excessive thread creation, poor memory locality, or incorrect synchronization. Effective optimization begins with measuring the relevant operating-system behavior, identifying the root cause, applying a targeted change, and validating the result against both performance and correctness. Deadlock prevention requires disciplined

resource ordering; starvation requires a fairness policy; and synchronization code must preserve invariants while reliably waking blocked workers.

Rubrics for Calculation

Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation